11/25/2026

schwarz digits



TypeScript 101

A practical workshop to learn the basics of TypeScript

by Mark Brockhoff



Who am I?

Name:
Mark Brockhoff

Position:
Tech Lead @ Schwarz Digits

Experience:
> 8 years as Full Stack Developer, > 6 of them using TypeScript

Hobbies:

- 🎷Playing saxophone in a band
- 🛩️Traveling
- 💡Small embedded projects e.g. IoT, Home automation

Agenda

schwarz digits

01 What is TypeScript and why does it exist?

02 Basics of the TS programming language

03 Hands on session 1

04 Working with async / await

05 Hands on session 2

06 Type narrowing & error handling

07 Hands on session 3

08 Wrap up / Q&A

schwarz digits

What is TypeScript and why does it exist?

01

schwarz digits

A look at the history of TypeScript

In the early 2010s web apps got more and more complex with Apps like **Gmail** and **Google Maps**

- JavaScript was designed for small scripts not huge enterprise applications
- The missing type system lead to bad scalability as the code base grew larger, e.g. due to the lack of good tooling
- Errors might only be caught at runtime possibly by the end user

Anders Hejlsberg at Microsoft designed TypeScript to solve this problem

- TypeScript is designed as a “Superset” of JavaScript -> JS code is also valid TS code
- Open Source from the beginning, since the first release in 2012
- Google later decided to rewrite v2 of the popular framework Angular in TypeScript

Typescript today

- According to the 2026 Stack Overflow survey TS is the 6th most popular programming language, slowly catching up to number 1 -> JS
- Thanks to a huge community, ecosystem and tooling TS is the de-facto standard for web but also heavily used for servers as well as desktop and mobile applications

schwarz digits

What's the advantage of TS?

JavaScript

```
const foo = 'bar';
foo = 42;
```

TypeScript

```
const foo: string = 'bar';
foo = 42;
```

=> TypeScript allows us to catch errors at build time

=> It allows us to model our code in a way where undefined states can't exist

=> It improves our developer experience with great tooling e.g. intellisense

How powerful is TypeScripts type system?

TypeScripts type system is basically a functional programming language by itself. Allowing you to build really smart types, e.g.:

- Allowing only capitalized strings to be passed to a function (Example)
- Having a variable which could have multiple different types (Example)
- Making sure a regex pattern is valid (e.g. ArkRegex)
- Adding autocomplete to GraphQL or SQL queries and typing the response (e.g. gql.tada or Drizzle)
- Executing a bubble sort in the type system (e.g. Github)
- Or even running Doom only using types: Youtube: Typescript types can run doom



Basics of the TypeScript programming language

02

schwarz digits

Syntax basics

Runtime

- Constants and variables
- Primitive values
- Objects and arrays
- Functions
- Conditions & Loops

Build time

- Defining types
- Deriving types
- Composing types

The official TypeScript docs contain very useful cheat sheets in case you want to look up some syntax:
https://www.typescriptlang.org/cheatsheets/

schwarz digits

Let’s get practical

Hands on session 1

03

What we'll be building today

Example App & repo setup

1. Clone the Repository from Github

   `git clone https://github.com/markbrockhoff/typescript-trivia.git`

2. Open it in an IDE / text editor of your choice
3. Open a terminal and install required dependencies (Node JS should be preinstalled):

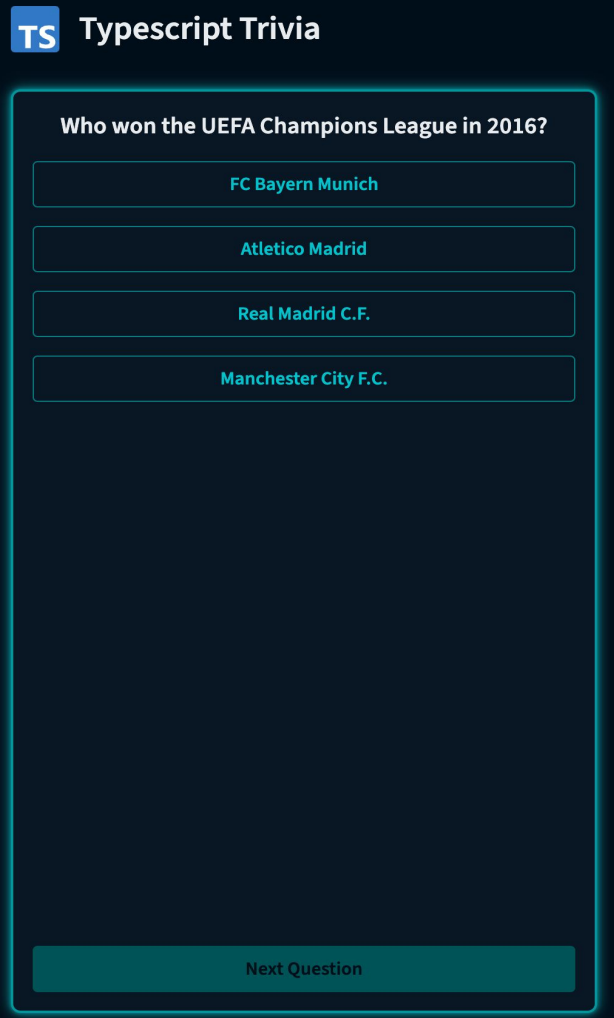
   `npm install`

4. Start the app by running:

   `npm run dev`

5. You browser should open http://localhost:5173/
   (You should see a "not implemented" error)





Task 1

Implement the static question provider



=> We'll have a Q&A after each Task

TypeScript Syntax Cheat Sheet

schwarz digits

1. Variable Declarations

```
// Immutable constant
const count: number = 42;

// Reassignable variable
let username: string = "Alice";

// Explicit type annotation
let isDone: boolean = true;
```

2. Primitive & Complex Types

```
// Arrays & Tuples
let nums: number[] = [1, 2];
let pair: [string, number] = ["a",
1];

// Objects
let user: User = {
  name: "Bob"
};
```

3. Functions & Signatures

```
// Function with typed params/return
function add(a: number, b: number) {
  return a + b;
}

// Arrow function
const log = (msg: string) => {
  console.log(msg);
};
```

4. Defining Types & Interfaces

```
// Custom Type Alias
type ID = string | number;

// Interface definition
interface User {
  readonly id: ID;
  name: string;
  role?: string; // Optional property
}
```

5. Unions & Intersections

```
// Union Types (OR)
type Status = "pending"|"approved";

// Intersection Types (AND)
type Admin = User & {
  permissions: string[]
};

// Type Narrowing
if (typeof id === "string") { ... }
```

6. Generics & Utility Types

```
// Generic Function
function wrap<T>(val: T): { item: T }
{
  return { item: val };
}

// Built-in Utility Types
type PartialUser = Partial<User>;
type UserKeys = keyof User;
```

Task 1

Shuffle the answers of the question by implementing the Fisher-Yates shuffle algorithm

schwarz digits

Working with async / await

04

Promises

Demo

- Promises are values which will be computed concurrently -> not in parallel
- Functions marked with the **async** keyword always return promises
- The **await** keyword can be used to stop code execution until the given promise is fulfilled
- Promises can **reject** e.g. if something inside them throws an exception
    - Be aware that every use of await could throw

schwarz digits

Fetching questions from an API

Hands on session 2

05

Task 2

Fetch questions dynamically from an API

Task 2 Cheat Sheet

Async / Await & Fetch API Reference

schwarz digits

1. Fetching API Data (HTTP GET)

```
// Define API response shape
type ApiResponse = {
  results: { question: string; correct_answer: string;
incorrect_answers: string[] }[];
};

// Async fetch & JSON parsing
const fetchQuestion = async () => {
  const res = await fetch('https://pokeapi.deno.dev' );
  const data: ApiResponse = await res.json();
};
```

2. Error Handling (Try / Catch)

```
// Handle network or promise rejections
const loadData = async () => {
  try {
    const data = await getPokemon(152);
    console.log(data.name);
  } catch (error) {
    // Handle error (e.g. log or fallback)
    console.error("Failed to fetch:", error);
  }
};
```

3. Mapping & Data Transformation

```
// Simple array transformation
const nums = [1, 2, 3];
const doubled = nums.map(x => x * 2);   // [2, 4, 6]

// Mapping objects to a new shape
const items = [{ id: 1, text: 'apple' }];
const formatted = items.map((item, index) => ({
  key: item.id,
  label: item.text.toUpperCase(),
  isFirst: index === 0
}));
```

4. Key Concepts & Pitfalls

- **async functions always return a Promise**
  Return types are automatically wrapped in Promise<T>.
- **await pauses execution**
  Stops code flow inside the async block until promise resolves.
- **res.json() is also asynchronous**
  Always use 'await res.json()' to parse the body stream.
- **decodeURIComponent() for encoded strings**
  Converts URL-encoded API text (like %20 or %27) into clean text.

schwarz digits

Type narrowing & error handling

06

schwarz digits

Type narrowing

Demo

- When working with polymorphic data or data from untrusted sources we need to **validate** it at runtime to ensure it matches the type we expect
- In TS this is done by **narrowing** the type of a value through conditions
- If the value is too dynamic, e.g. if it’s an array, a **type guard** needs to be created to perform this validation for TS
- In larger projects it may be worth it to add a validation library to make this simpler.
  E.g.: Zod, ArkType or Valibot

schwarz digits

Error handling

Demo

- TS supports **exceptions**, if the program runs into an issue it can **throw** an instance of the **Error** class
- Thrown errors can be caught by using a **try/catch** block
- It’s possible to create **custom errors** by extending the Error class, e.g. to store custom data about the error inside them
- When handling an error inside a catch block, **narrowing** is required to access its properties

schwarz digits

Type safety everywhere

Hands on session 3

07

Task 3

Use type narrowing to validate the API response



schwarz digits

Type Narrowing Cheat Sheet

1. Basic Checks (typeof, truthy, in)

```ts
const test: unknown = { ... };
// 1. Check basic type
if (typeof test !== "object") return;
// 2. Check truthiness (not null)
if (!test) return;
// 3. Property existence check
if (!("bar" in test)) return;
```

2. Complex Structures & Arrays

```ts
// Check if property is an Array
if (!test.bar || !Array.isArray(test.bar)) return;
// Narrow intermediate array elements
const obj: unknown = test.bar[0];
if (!obj || typeof obj !== "object") return;
if (!("bizz" in obj)) return;
// Safe access: obj.bizz is now narrowed!
```

3. Custom Type Guards (Type Predicates: `arg is Type`)

```ts
// Type Guard function returns a predicate ("item is { bizz: number }")
const isBizzObject = (item: unknown): item is { bizz: number } => {
  return !!item && typeof item === "object" && "bizz" in item && typeof item.bizz === "number";
};
// Array Type Guard using .every()
const isBizzArray = (arr: unknown[]): arr is { bizz: number }[] => arr.every(isBizzObject);
// Usage: TS automatically narrows inside the if-block
if (isBizzArray(test.bar)) { console.log(test.bar[].bizz); }
```

Task 3

Add error handling and retries

Error Handling Cheat Sheet

schwarz digits

1. Defining Custom Errors

```
// Extend native Error class
class PokeApiError extends Error {
  pokemonId: number;
  constructor(pokemonId: number) {
    super(`Failed: ${pokemonId}`);
    this.pokemonId = pokemonId;
  }
}
```

2. Throwing Exceptions

```
const getPokemon = async (id: number) => {
  const res = await fetch(`.../${id}`);
  // Check non-200 status and throw
  if (res.status !== 200) {
    throw new PokeApiError(id);
  }
  return await res.json();
};
```

3. Catching & Narrowing Unknown Errors (try / catch / instanceof)

```
try {
  const pokemon = await getPokemon(-1);
} catch (error) {
  // error is of type 'unknown' by default; narrow before accessing properties
  if (error instanceof PokeApiError) {
    console.error(error.message);  // Narrowed to PokeApiError!
  } else {
    console.error("Unknown error");
  }
}
```

schwarz digits

Wrap up / Q&A

08

schwarz digits

Good reads for recap / to go further

- Bad practices in TypeScript
- The TypeScript Handbook (part of the official TS docs)
- Total TypeScript Essentials by Matt Pocock
- https://learntypescript.dev/

schwarz digits

Feedback

Please take a short moment to fill out the survey. It's 100% anonymous and helps me improve, thank you.



schwarz digits

Let’s stay in touch





https://schwarz-digits.de